

智能家庭控制系統

專題進度報告—RPi Server 開發進展

趙子倫 詹詠翔 詹育晟

2026 年 5 月

基於 Raspberry Pi 3 與 STM32 之 BLE 通訊應用

專案動機與目標

系統架構

RPi Server 設計

已完成進度

進行中與規劃

結語

專案動機與目標

動機：異質設備整合的挑戰

- STM32 開發板具備豐富感測器，但網路與伺服能力薄弱
- 若每個節點各自連網，將造成部署成本高且耗電
- 需要一個通用的中央伺服器動態管理多個感測節點

設計目標

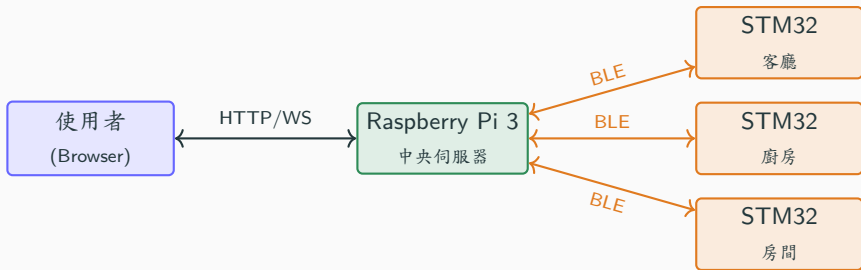
建立一套高度模組化、具備本地端安全性、以 BLE 為通訊核心的智慧家庭生態系統。

核心架構：中央伺服器 + BLE

- 使用者透過 Web UI 遠端控制
- 周邊設備不需各自連網，統一透過 BLE 與 RPi 通訊
- 達到顯著省電與部署簡化效果

系統架構

硬體部署：一對多分散式節點



- RPi 同時管理多個不同空間的 STM32 節點
- 使用者端僅需與 RPi 互動

三階段通訊流程

廣播 (Advertising) → 連線 (GATT Connect) → 推播 (Notify)

頻道類型	BLE 行為	應用情境
Controller	Server → Write	燈光開關、溫度設定
Display	Server ← Read/Notify	溫濕度、晃動、聲音偵測

註：未來預留 Hybrid 複合型頻道擴充空間。

RPi Server 設計

執行環境

- Python 3.11+ (dev 用 3.12)
- uv 套件管理
- Taskfile 統一指令入口

Web 層

- Flask + Flask-Login
- Flask-WTF (CSRF)
- Jinja2 模板

資料層

- SQLite + WAL
- 自製 repository 層

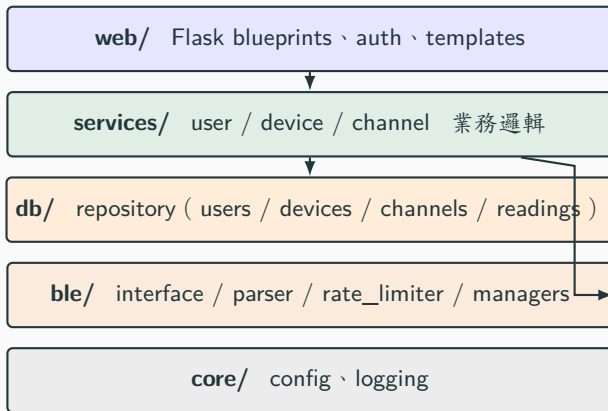
BLE 層

- bluepy (Linux/RPi)
- Mock 實作 (Windows dev)

品質工具

- ruff、mypy strict
- pytest (103 tests)

模組分層



- 上層只依賴下層；service 層為純業務邏輯
- BLE 介面用 Protocol 抽象，方便 mock 測試

已完成進度

Phase 1 : 骨架

- uv 專案、目錄結構、config.py 讀環境變數
- db/schema.sql : users / devices / channels / readings + WAL
- __main__.py : 可啟動 Flask app , 含 /health

Phase 2 : BLE 通訊

- ble/interface.py : BLEManager Protocol 抽象
- ble/parser.py : 10 種格式雙向轉換 (int/uint 8/16/32、float32 , LE/BE)
- ble/rate_limiter.py : per-key 限頻 , 可注入 fake clock
- ble/mock_manager.py / bluepy_manager.py : Windows dev 與 RPi 雙實作
- bluepy_manager : 每個 peripheral 一個 worker thread + command queue , 避開 bluepy 跨執行緒問題

四個 repository 模組，每個自帶領域錯誤型別：

- `db/users.py` : `create` / `get_by_id` / `get_by_username` / `update_password`
- `db/devices.py` : `create` / `get_by_*` / `list_*` / `delete`
- `db/channels.py` : 含 `Literal["controller", "display"]` 型別
- `db/readings.py` : `insert` / `list_by_channel` / `delete_older_than`

設計重點

- 時間戳統一 UTC "YYYY-MM-DD HH:MM:SS" 字串
- 領域錯誤 : `DuplicateUsernameError`、`DeviceNotFoundError` ...
- 測試 fixture : in-memory SQLite + schema 已套用

UserService

- bcrypt 雜湊 (cost 可注入，測試用低 cost)
- 密碼長度 8-72 bytes (避免 bcrypt 截斷)

DeviceService (建構注入 BLEManager)

- scan / add_device / remove_device / list_devices
- add_device : 驗證 MAC → 寫 DB → best-effort 連線

ChannelService (注入 BLE、RateLimiter、on_reading callback)

- write_command : 僅 controller 型，編碼後 BLE write
- handle_notify : 解析 → 即時推播 (不限頻) → DB 寫入 (限頻)
- get_history / list_by_device

- `web/__init__.py` : application factory `create_app`
- 整合 **Flask-Login** + **Flask-WTF CSRF**
- `web/db.py` : per-request SQLite 連線 (`flask.g` + `teardown`)
- `web/auth.py` : `/auth/register`、`/auth/login`、`/auth/logout`
- `next` 參數的 **open-redirect** 防護
- Jinja2 陽春模板 (`base` / `index` / `auth`) 含 CSRF token

里程碑

可從瀏覽器完成「註冊 → 自動登入 → 登出」完整流程

測試覆蓋

- **103** unit tests passing
- in-memory SQLite fixture
- fake clock 注入測限頻
- mock BLEManager 測 service

CI 入口

`task ci = lint + typecheck + test`

靜態檢查

- `ruff check` 全綠
- `mypy src (strict)` 全綠
- per-module type overrides

進行中與規劃

尚待補齊的 Blueprint :

- `/devices` : 列表、掃描、新增、刪除
- `/devices/<id>/channels` : 新增、刪除
- `POST /channels/<id>/write` : 控制型頻道下指令

設計重點

- 重用既有 `DeviceService` / `ChannelService`
- Server-rendered 表單 + CSRF ; 後續再以 HTMX 強化體驗

後端

- Flask-SocketIO (threading 模式)
- 與 bluepy worker thread 串接
- 每個監控型頻道一個 SocketIO room

前端

- Jinja2 + **HTMX** 漸進增強
- **Chart.js** 繪製歷史趨勢圖
- 訂閱 Notify 事件後即時更新圖表

硬體：B-L475E-IOT01A

- HTS221 溫濕度感測器
- LSM6DSL 加速度 / 陀螺儀
- MP34DT01 數位麥克風
- NFC、使用者 LED

韌體任務

- 開發 BLE GATT Service / Characteristic 設計
- 感測器資料採集與封包格式（對齊 RPi 端 parser）
- 接收控制指令（LED、設定值）
- 工具鏈：**STM32CubeIDE** + ST HAL

- 多 STM32 節點**分散式佈署測試**（客廳 / 廚房 / 房間）
- RPi ↔ STM32 **真機聯調**
- BLE **Notify 推播延遲**量測
- 異常情境：斷線重連、信號弱、節點重啟

驗收標準

至少 3 個 STM32 節點同時連線 30 分鐘以上，Notify 訊息掉包率 < 1%。

- systemd service 安裝腳本 (boot 即啟動)
- RPi 上長時間穩定性測試 (記憶體 / 連線 / DB 大小)
- 部署文件與 troubleshooting guide
- 成果展示準備

結語

- ✓ **Phase 1-2** : 骨架 + BLE 通訊鏈結
- ✓ **Phase 3a-c** : DB / Service / 認證 103 tests 全綠
 - **Phase 3d** : Device / Channel CRUD
 - **Phase 3e** : SocketIO + 前端趨勢圖
 - **STM32 韌體** : B-L475E-IOT01A BLE peripheral
 - **Phase 4** : 整合測試與部署

下一步： 完成 Phase 3d，並開始 STM32 端 BLE Service 設計。

Q & A